

Practical-4

Problem Definition

Implement and evaluate two different data integration pipelines using identical retail transaction data. The first pipeline follows the ETL (Extract, Transform, Load) architecture, where raw data is extracted from local storage, transformed in application memory using Pandas, and then loaded into a DuckDB analytical database.

The second pipeline follows the ELT (Extract, Load, Transform) architecture, where raw transaction data is first loaded directly into a DuckDB staging table and transformations are then performed using the DuckDB SQL engine.

The practical compares both architectures based on execution time, processing efficiency, data lineage, transformation recovery, resource utilization, and maintenance requirements.

Dataset

The dataset consists of high-volume, un-normalized retail sales transaction records.

The raw dataset contains 100,000 retail transaction records with fields such as:

- Transaction ID
- Store Code
- Product
- Category
- Quantity
- Price
- Sale Date

The dataset intentionally contains data-quality inconsistencies such as:

- Leading and trailing white spaces
- Inconsistent category capitalization
- Legacy store codes
- Raw numeric values
- Unstandardized product names

ETL Architecture

The ETL pipeline follows:

Raw CSV File



Extract



Pandas DataFrame



Transform



Cleaned Data



DuckDB



Final Table

ELT Architecture

The ELT pipeline follows:

Raw CSV File



Extract



DuckDB Staging Table

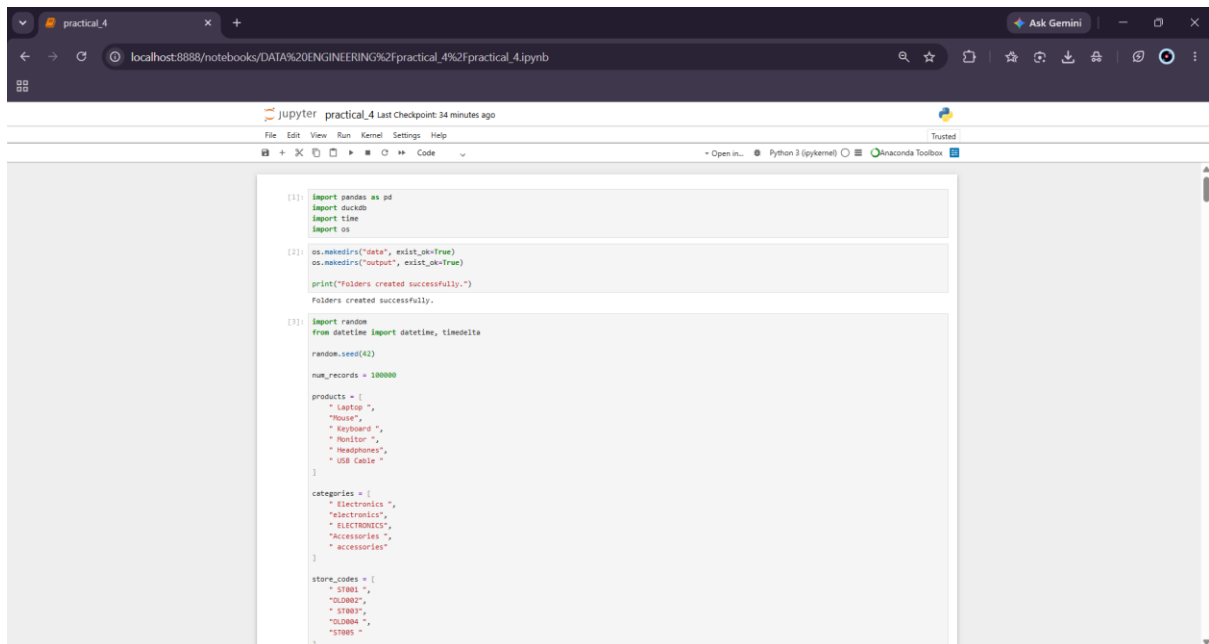


Transform using SQL



Final DuckDB Table

OUTPUT/SCREENSHOTS



The screenshot shows a Jupyter Notebook interface with a dark theme. The browser address bar indicates the notebook is located at `localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb`. The notebook title is "practical_4" and it was last checkpointed 34 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations, running the notebook, and switching kernels. The code area contains three cells:

```
[1]: import pandas as pd
import duckdb
import time
import os

[2]: os.makedirs("data", exist_ok=True)
os.makedirs("output", exist_ok=True)
print("Folders created successfully.")
Folders created successfully.

[3]: import random
from datetime import datetime, timedelta

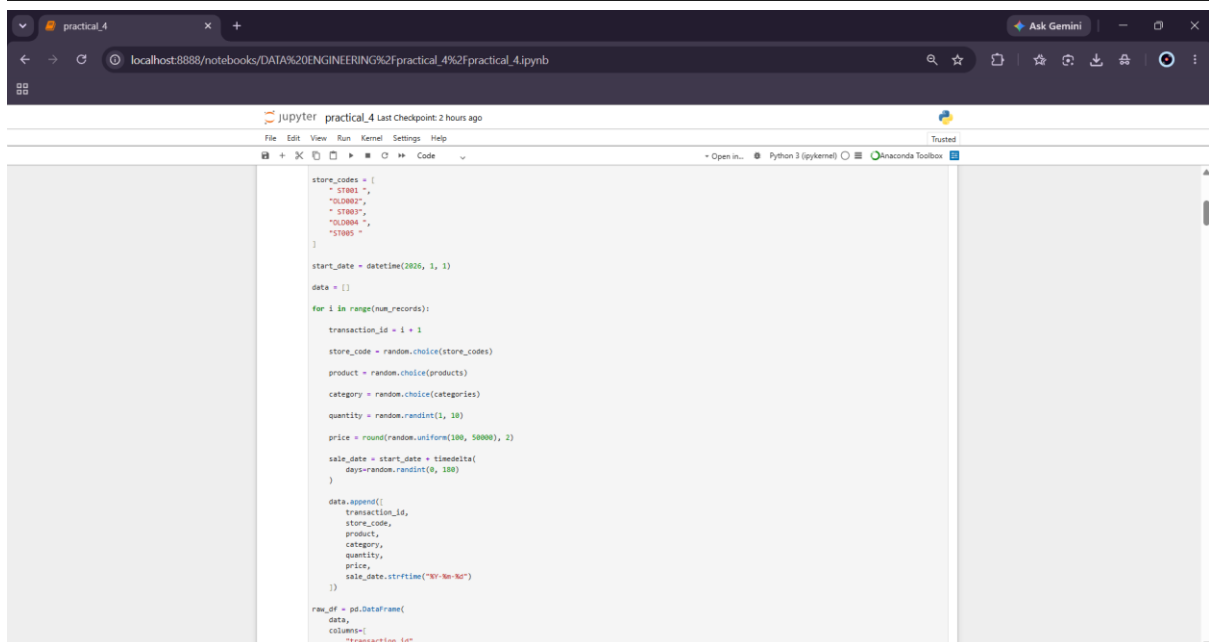
random.seed(42)

num_records = 100000

products = [
    "Laptop ",
    "Mouse ",
    "Keyboard ",
    "Monitor ",
    "Headphones",
    "USB Cable "
]

categories = [
    "Electronics ",
    "Electronics",
    "ELECTRONICS",
    "Accessories ",
    "Accessories"
]

store_codes = [
    "ST001 ",
    "OLD002",
    "ST003",
    "OLD004 ",
    "ST005 "
]
```



This screenshot shows the continuation of the Jupyter Notebook. The code area contains the following cells:

```
store_codes = [
    "ST001 ",
    "OLD002",
    "ST003",
    "OLD004 ",
    "ST005 "
]

start_date = datetime(2020, 1, 1)

data = []

for i in range(num_records):
    transaction_id = i + 1
    store_code = random.choice(store_codes)
    product = random.choice(products)
    category = random.choice(categories)
    quantity = random.randint(1, 10)
    price = round(random.uniform(100, 50000), 2)
    sale_date = start_date + timedelta(
        days=random.randint(0, 100)
    )

    data.append({
        transaction_id,
        store_code,
        product,
        category,
        quantity,
        price,
        sale_date.strftime("%Y-%m-%d")
    })

raw_df = pd.DataFrame(
    data,
    columns=[
        "transaction_id",
```

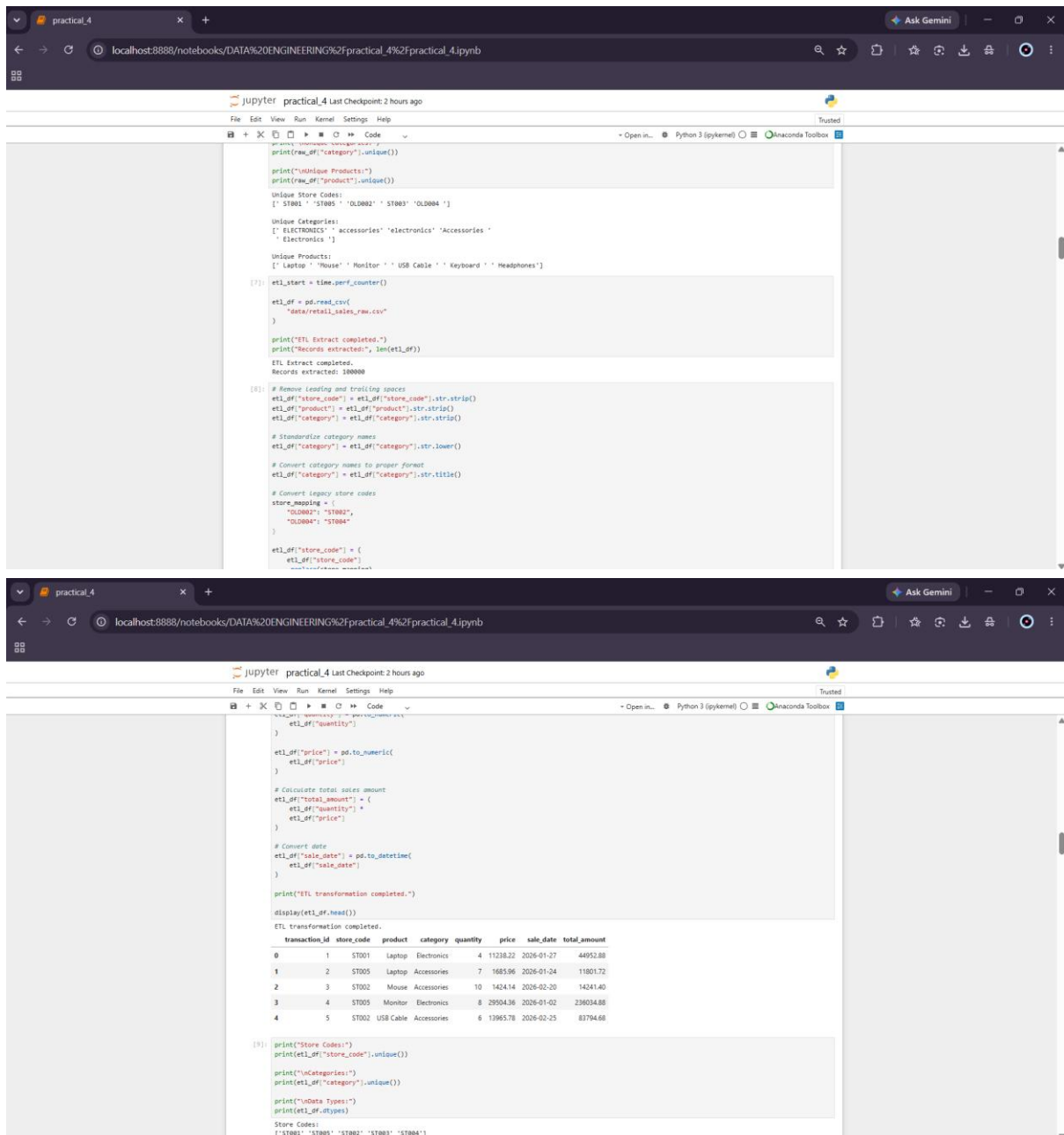
The image displays two screenshots of a Jupyter Notebook interface, likely running on a local server at localhost:8888. The notebook is titled "practical_4" and shows a series of code cells and their outputs.

Top Screenshot:

- The first code cell defines a list of columns: `data, columns=["transaction_id", "store_code", "product", "category", "quantity", "price", "sale_date"]`.
- The second code cell uses `raw_df.head()` to display the first 5 rows of the dataset.
- The output shows a table with columns: `transaction_id`, `store_code`, `product`, `category`, `quantity`, `price`, and `sale_date`.
- The third code cell saves the dataset to a CSV file: `raw_df.to_csv("data/retail_sales_raw.csv", index=False)`.
- The output confirms the save operation: "Raw dataset saved. Number of records: 100000".
- The fourth code cell prints the dataset shape and column information: `print(raw_df.shape)`, `print(raw_df.info())`, and `print(raw_df.head(10))`.
- The output shows the dataset shape as (100000, 7) and provides detailed column information.

Bottom Screenshot:

- The first code cell prints the unique store codes: `print("Unique Store Codes:")`, `print(raw_df["store_code"].unique())`.
- The second code cell prints the unique categories: `print("Unique Categories:")`, `print(raw_df["category"].unique())`.
- The output shows the unique store codes and categories.



The image displays two screenshots of a Jupyter Notebook interface, showing the progression of data extraction and transformation (ETL) tasks.

Top Screenshot: The notebook is titled "practical_4" and shows the initial data extraction step. The code reads a CSV file named "data/retail_sales_raw.csv" and prints the first few rows. The output shows the raw data with columns: transaction_id, store_code, product, category, quantity, price, sale_date, and total_amount. The data is then cleaned and transformed into a structured format.

```
print(pd.read_csv('data/retail_sales_raw.csv'))
print(raw_df['category'].unique())
print("\nUnique Products:")
print(raw_df['product'].unique())

Unique Store Codes:
['ST001' 'ST005' 'OL0001' 'ST003' 'OL0004']

Unique Categories:
['Electronics' 'Accessories' 'Electronics' 'Accessories'
 'Electronics']

Unique Products:
['Laptop' 'Mouse' 'Monitor' 'USB Cable' 'Keyboard' 'Headphones']

[7]: etl_start = time.perf_counter()

etl_df = pd.read_csv(
    "data/retail_sales_raw.csv"
)

print("ETL Extract completed.")
print("Records extracted:", len(etl_df))
ETL Extract completed.
Records extracted: 100000

[8]: # Remove leading and trailing spaces
etl_df["store_code"] = etl_df["store_code"].str.strip()
etl_df["product"] = etl_df["product"].str.strip()
etl_df["category"] = etl_df["category"].str.strip()

# Standardize category names
etl_df["category"] = etl_df["category"].str.lower()

# Convert category names to proper format
etl_df["category"] = etl_df["category"].str.title()

# Convert legacy store codes
store_mapping = {
    "OL0001": "ST002",
    "OL0004": "ST004"
}

etl_df["store_code"] = (
    etl_df["store_code"].
```

Bottom Screenshot: The notebook continues with the transformation step. The code calculates the total sales amount for each transaction and converts the sale date to a datetime format. The output shows the transformed data with columns: transaction_id, store_code, product, category, quantity, price, sale_date, and total_amount. The data is then displayed as a table.

```
etl_df["quantity"] = etl_df["quantity"]
etl_df["price"] = pd.to_numeric(
    etl_df["price"]
)

# Calculate total sales amount
etl_df["total_amount"] = (
    etl_df["quantity"] *
    etl_df["price"]
)

# Convert date
etl_df["sale_date"] = pd.to_datetime(
    etl_df["sale_date"]
)

print("ETL transformation completed.")
display(etl_df.head())

ETL transformation completed.

transaction_id  store_code  product  category  quantity  price  sale_date  total_amount
0              1      ST001   Laptop  Electronics      4    11238.22  2026-01-27      44952.88
1              2      ST005   Laptop  Accessories     7     1685.96  2026-01-24      11801.72
2              3      ST002   Mouse   Accessories    10     1424.14  2026-02-20      14241.40
3              4      ST005   Monitor  Electronics     8    29504.36  2026-01-02    236034.88
4              5      ST002  USB Cable  Accessories     6     13965.78  2026-02-25     83794.68

[9]: print("Store Codes:")
print(etl_df["store_code"].unique())

print("\nCategories:")
print(etl_df["category"].unique())

print("\nData Types:")
print(etl_df.dtypes)

Store Codes:
['ST001' 'ST005' 'ST002' 'ST003' 'ST004']
```

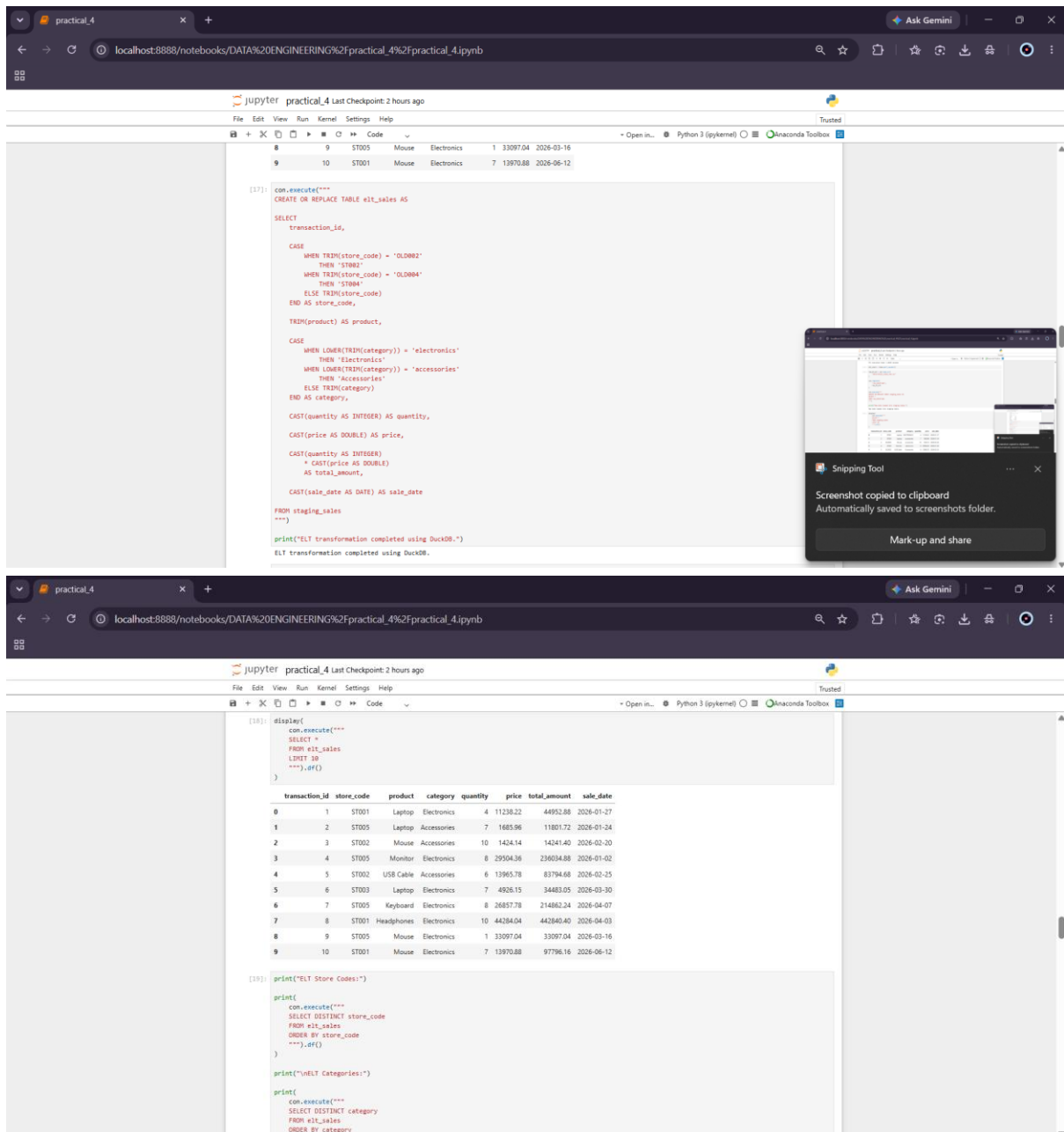
The image displays two screenshots of a Jupyter Notebook interface, likely running on a local host (localhost:8888). The notebook is titled "practical_4" and shows code for connecting to a DuckDB database, registering a table, and executing SQL queries.

Top Screenshot: The code defines store codes, categories, and data types. It connects to a DuckDB database, registers a table named "etl_dataframe", and executes a SQL query to create or replace a table named "etl_sales AS". The query selects data from "etl_dataframe". The code then prints the ETL data loaded into DuckDB, the ETL count, and the ETL execution time (1.0638 seconds).

Bottom Screenshot: The code continues with the ETL process, reading data from a CSV file, registering it as "raw_dataframe", and executing a SQL query to create or replace a table named "staging_sales AS". The query selects data from "raw_dataframe". The code then prints the raw data loaded into the staging table. The final output is a table showing the results of the ETL process, including transaction ID, store code, product, category, quantity, price, and sale date.

ETL execution time: 1.0638 seconds

transaction_id	store_code	product	category	quantity	price	sale_date	
0	1	ST001	Laptop	ELECTRONICS	4	11238.22	2026-01-27
1	2	ST005	Laptop	accessories	7	1685.96	2026-01-24
2	3	OLD002	Mouse	accessories	10	1424.14	2026-02-20
3	4	ST005	Monitor	electronics	8	29504.36	2026-01-02
4	5	OLD002	USB Cable	Accessories	6	13965.78	2026-03-15
5	6	ST003	Laptop	Electronics	7	4926.15	2026-03-30
6	7	ST005	Keyboard	Electronics	8	26857.78	2026-04-07
7	8	ST001	Headphones	ELECTRONICS	10	44284.04	2026-04-03



The screenshot displays a Jupyter Notebook interface with the following content:

Code Cell [27]:

```
con.execute("""
CREATE OR REPLACE TABLE elt_sales AS

SELECT
    transaction_id,

    CASE
        WHEN TRIM(store_code) = 'OLD002'
        THEN 'ST001'
        WHEN TRIM(store_code) = 'OLD004'
        THEN 'ST004'
        ELSE TRIM(store_code)
    END AS store_code,

    TRIM(product) AS product,

    CASE
        WHEN LOWER(TRIM(category)) = 'electronics'
        THEN 'Electronics'
        WHEN LOWER(TRIM(category)) = 'accessories'
        THEN 'Accessories'
        ELSE TRIM(category)
    END AS category,

    CAST(quantity AS INTEGER) AS quantity,

    CAST(price AS DOUBLE) AS price,

    CAST(quantity AS INTEGER)
    * CAST(price AS DOUBLE)
    AS total_amount,

    CAST(sale_date AS DATE) AS sale_date

FROM staging_sales
""")

print("ELT transformation completed using DuckDB.")
ELT transformation completed using DuckDB.
```

Output:

	transaction_id	store_code	product	category	quantity	price	total_amount	sale_date
8	9	ST005	Mouse	Electronics	1	33097.04	33097.04	2026-03-16
9	10	ST001	Mouse	Electronics	7	13970.88	97796.16	2026-06-12

Code Cell [28]:

```
display(
    con.execute("""
SELECT *
FROM elt_sales
LIMIT 10
""").df()
)
```

Output:

	transaction_id	store_code	product	category	quantity	price	total_amount	sale_date
0	1	ST001	Laptop	Electronics	4	11236.22	44952.88	2026-01-27
1	2	ST005	Laptop	Accessories	7	1685.96	11801.72	2026-01-24
2	3	ST002	Mouse	Accessories	10	1424.14	14241.40	2026-02-20
3	4	ST005	Monitor	Electronics	8	29504.36	236034.88	2026-01-02
4	5	ST002	USB Cable	Accessories	6	13965.78	83794.68	2026-02-25
5	6	ST003	Laptop	Electronics	7	4926.15	34483.05	2026-03-30
6	7	ST005	Keyboard	Electronics	8	26857.78	214862.24	2026-04-07
7	8	ST001	Headphones	Electronics	10	44284.04	442840.40	2026-04-03
8	9	ST005	Mouse	Electronics	1	33097.04	33097.04	2026-03-16
9	10	ST001	Mouse	Electronics	7	13970.88	97796.16	2026-06-12

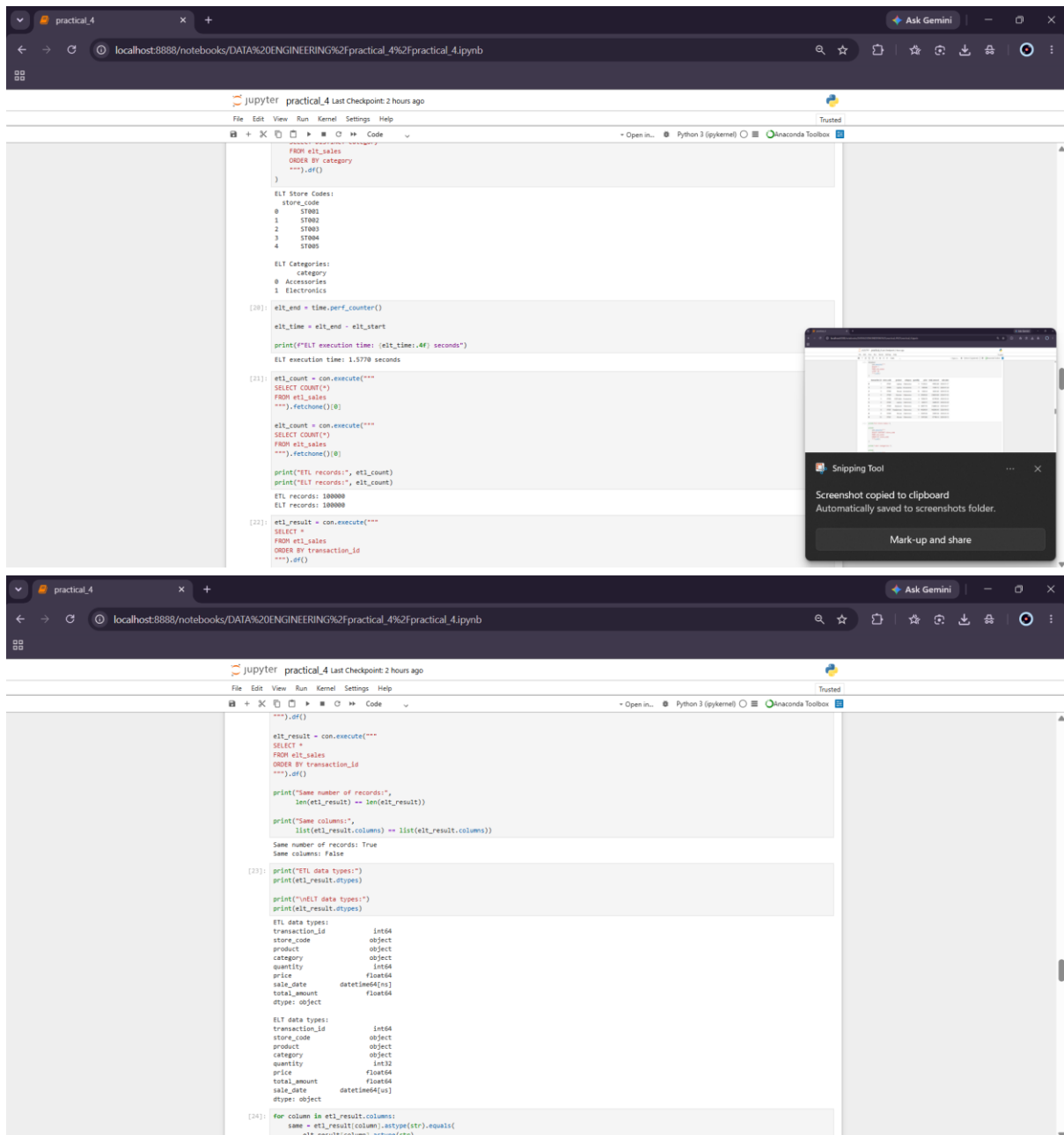
Code Cell [29]:

```
print("ELT Store Codes:")

print(
    con.execute("""
SELECT DISTINCT store_code
FROM elt_sales
ORDER BY store_code
""").df()
)

print("\nELT Categories:")

print(
    con.execute("""
SELECT DISTINCT category
FROM elt_sales
ORDER BY category
""").df()
)
```



```
practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel) Anaconda Toolbox

FROM etl_sales
ORDER BY category
).df()

}

ETL Store Codes:
store_code
0 ST001
1 ST002
2 ST003
3 ST004
4 ST005

ETL Categories:
category
0 Accessories
1 Electronics

[20]: etl_end = time.perf_counter()

etl_time = etl_end - etl_start
print(f"ETL execution time: {etl_time:.4f} seconds")

ETL execution time: 1.5770 seconds

[21]: etl_count = con.execute("""
SELECT COUNT(*)
FROM etl_sales
""").fetchone()[0]

etl_count = con.execute("""
SELECT COUNT(*)
FROM etl_sales
""").fetchone()[0]

print("ETL records:", etl_count)
print("ETL records:", etl_count)

ETL records: 100000
ETL records: 100000

[22]: etl_result = con.execute("""
SELECT *
FROM etl_sales
ORDER BY transaction_id
""").df()

""").df()

etl_result = con.execute("""
SELECT *
FROM etl_sales
ORDER BY transaction_id
""").df()

print("Same number of records:",
len(etl_result) == len(etl_result))

print("Same columns:",
list(etl_result.columns) == list(etl_result.columns))

Same number of records: True
Same columns: False

[23]: print("ETL data types:")
print(etl_result.dtypes)

print("ETL data types:")
print(etl_result.dtypes)

ETL data types:
transaction_id    int64
store_code        object
product           object
category          int64
quantity          float64
price             float64
sale_date         datetime64[ns]
total_amount      float64
dtype: object

ETL data types:
transaction_id    int64
store_code        object
product           object
category          int64
quantity          float64
price             float64
sale_date         datetime64[ns]
total_amount      float64
dtype: object

[24]: for column in etl_result.columns:
same = etl_result[column].astype(str).equals(
etl_result[column].astype(str))
```

The image displays two screenshots of a Jupyter Notebook interface, likely running on a local host (localhost:8888). The notebook is titled "practical_4" and shows a series of code cells for data processing and comparison.

Top Screenshot: The code cell contains the following Python code:

```
same = etl_result[column].astype(str).equals(
    elt_result[column].astype(str)
)

print(column, "-", same)

transaction_id : True
store_code : True
product : True
category : True
quantity : True
price : True
sale_date : True
total_amount : True

[25]: # Make copies
etl_final = etl_result.copy()
elt_final = elt_result.copy()

# Use the same column order for both DataFrames
common_columns = sorted(
    set(etl_final.columns) & set(elt_final.columns)
)

etl_final = etl_final[common_columns]
elt_final = elt_final[common_columns]

# Reset row indexes
etl_final = etl_final.reset_index(drop=True)
elt_final = elt_final.reset_index(drop=True)

# Convert values to strings for logical comparison
etl_final = etl_final.astype(str)
elt_final = elt_final.astype(str)

print("ETL columns:")
print(list(etl_final.columns))

print("ELT columns:")
print(list(elt_final.columns))

print("Same number of records:",
      len(etl_final) == len(elt_final))

print("Same columns:",
      list(etl_final.columns) == list(elt_final.columns))
```

Bottom Screenshot: The code cell contains the following Python code:

```
print("Same columns:",
      list(etl_final.columns) == list(elt_final.columns))

print("ETL and ELT data values identical:",
      etl_final.equals(elt_final))

ETL columns:
['category', 'price', 'product', 'quantity', 'sale_date', 'store_code', 'total_amount', 'transaction_id']

ELT columns:
['category', 'price', 'product', 'quantity', 'sale_date', 'store_code', 'total_amount', 'transaction_id']

Same number of records: True
Same columns: True
ETL and ELT data values identical: True

[26]: performance = pd.DataFrame({
    "Pipeline": ["ETL", "ELT"],
    "Execution Time (seconds)": [
        etl_time,
        elt_time
    ],
    "Records Processed": [
        etl_count,
        elt_count
    ]
})

display(performance)

Pipeline  Execution Time (seconds)  Records Processed
0    ETL                1.063848             100000
1    ELT                1.576983             100000

[27]: difference = abs(etl_time - elt_time)

print(f"ETL execution time: {etl_time:.4f} seconds")
print(f"ELT execution time: {elt_time:.4f} seconds")
print(f"Difference: {difference:.4f} seconds")

ETL execution time: 1.0638 seconds
ELT execution time: 1.5770 seconds
Difference: 0.5132 seconds

[28]: if etl_time < elt_time:
```

The screenshot shows a Jupyter Notebook interface with two cells of code and their outputs. The browser address bar shows the URL: `localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb`.

Cell 1:

```
[28]: if etl_time < elt_time:
      print("ETL was faster in this local benchmark.")
      elif etl_time < elt_time:
      print("ETL was faster in this local benchmark.")
      else:
      print("Both pipelines had the same execution time.")

      ETL was faster in this local benchmark.

[29]: etl_throughput = etl_count / etl_time
      elt_throughput = elt_count / elt_time

      print(f"ETL throughput: {etl_throughput:.2f} records/second")
      print(f"ETL throughput: {elt_throughput:.2f} records/second")

      ETL throughput: 93998.43 records/second
      ETL throughput: 63412.34 records/second

[31]: print("Staging table record count:")

      print(
      con.execute("""
      SELECT COUNT(*) AS record_count
      FROM staging_sales
      """).df()
      )

      Staging table record count:
      record_count
      0          100000

[32]: display(
      con.execute("""
      SELECT *
      FROM staging_sales
      LIMIT 10
      """).df()
      )
```

transaction_id	store_code	product	category	quantity	price	sale_date
0	1	ST001	Laptop	ELECTRONICS	4	11238.22 2026-01-27
1	2	ST005	Laptop	accessories	7	1685.96 2026-01-24
2	3	OLD002	Mouse	accessories	10	1424.14 2026-02-20

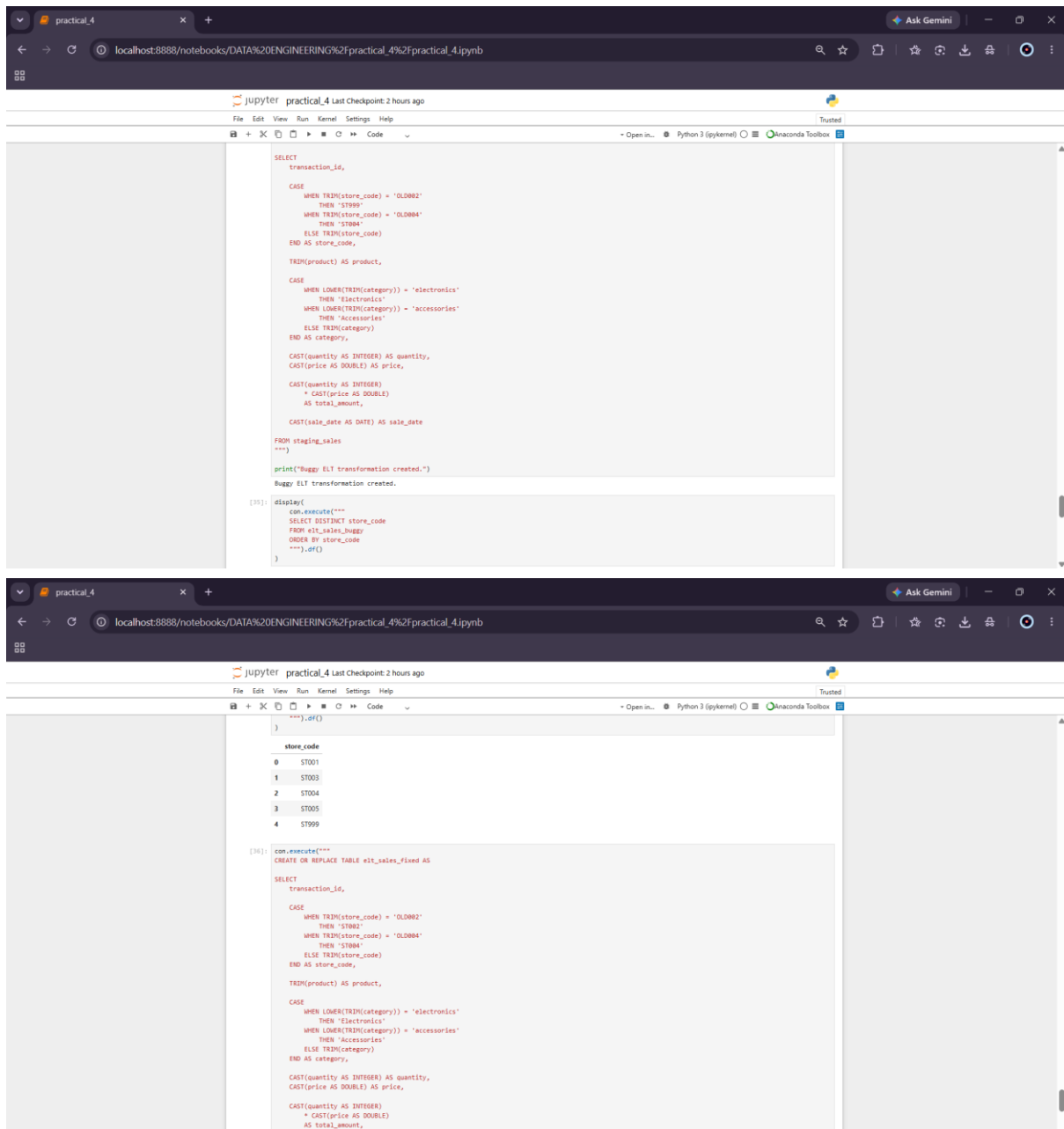
Cell 2:

```
[33]: display(
      con.execute("""
      SELECT *
      FROM elt_sales
      LIMIT 10
      """).df()
      )
```

transaction_id	store_code	product	category	quantity	price	total_amount	sale_date
3	4	ST005	Monitor	electronics	8	29504.36	2026-01-02
4	5	OLD002	USB Cable	Accessories	6	13965.78	2026-02-25
5	6	ST003	Laptop	Electronics	7	4926.15	2026-03-30
6	7	ST005	Keyboard	Electronics	8	26857.78	2026-04-07
7	8	ST001	Headphones	ELECTRONICS	10	44284.04	2026-04-03
8	9	ST005	Mouse	Electronics	1	33097.04	2026-03-16
9	10	ST001	Mouse	Electronics	7	13970.88	2026-06-12

Cell 3:

```
[34]: con.execute("""
      CREATE OR REPLACE TABLE elt_sales_huggy AS
      SELECT
```



The image displays two screenshots of a Jupyter Notebook interface, likely running on a local host (localhost:8888). The notebook is titled "practical_4" and shows SQL code for data transformation and its execution results.

Top Screenshot: The notebook displays a SQL query for creating a Buggy ELT transformation. The query selects data from a staging table, applies a CASE statement for store codes, and casts various fields. The output shows the transformation created successfully.

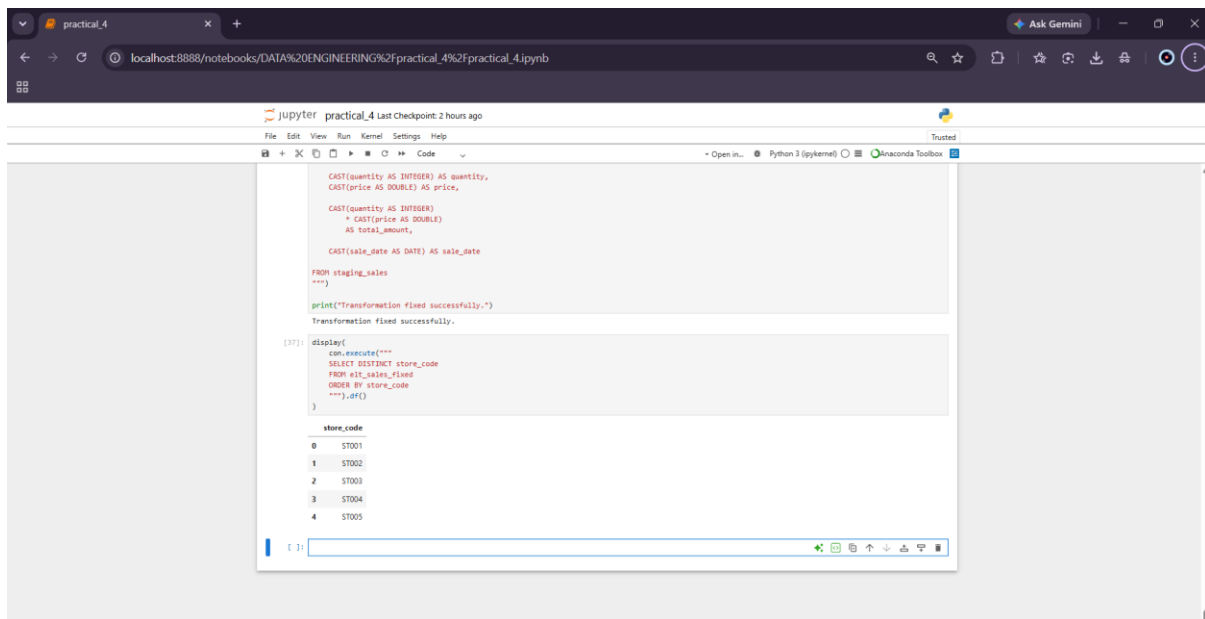
```
SELECT
  transaction_id,
  CASE
    WHEN TRIM(store_code) = 'OLD002'
    THEN 'ST001'
    WHEN TRIM(store_code) = 'OLD004'
    THEN 'ST004'
    ELSE TRIM(store_code)
  END AS store_code,
  TRIM(product) AS product,
  CASE
    WHEN LOWER(TRIM(category)) = 'electronics'
    THEN 'Electronics'
    WHEN LOWER(TRIM(category)) = 'accessories'
    THEN 'Accessories'
    ELSE TRIM(category)
  END AS category,
  CAST(quantity AS INTEGER) AS quantity,
  CAST(price AS DOUBLE) AS price,
  CAST(quantity AS INTEGER)
  * CAST(price AS DOUBLE)
  AS total_amount,
  CAST(sale_date AS DATE) AS sale_date
FROM staging_sales
)))
print("Buggy ELT transformation created.")
Buggy ELT transformation created.
```

Bottom Screenshot: The notebook displays the execution results of the SQL query. The output shows a list of store codes (ST001, ST003, ST004, ST005, ST009) and the execution of the Buggy ELT transformation, which creates or replaces the table elt_sales_fixed.

```
store_code
0      ST001
1      ST003
2      ST004
3      ST005
4      ST009

[36]: com.execute("""
CREATE OR REPLACE TABLE elt_sales_fixed AS

SELECT
  transaction_id,
  CASE
    WHEN TRIM(store_code) = 'OLD002'
    THEN 'ST002'
    WHEN TRIM(store_code) = 'OLD004'
    THEN 'ST004'
    ELSE TRIM(store_code)
  END AS store_code,
  TRIM(product) AS product,
  CASE
    WHEN LOWER(TRIM(category)) = 'electronics'
    THEN 'Electronics'
    WHEN LOWER(TRIM(category)) = 'accessories'
    THEN 'Accessories'
    ELSE TRIM(category)
  END AS category,
  CAST(quantity AS INTEGER) AS quantity,
  CAST(price AS DOUBLE) AS price,
  CAST(quantity AS INTEGER)
  * CAST(price AS DOUBLE)
  AS total_amount,
```



The screenshot shows a Jupyter Notebook titled 'practical_4' running on a local host. The code defines a transformation function that casts columns and calculates a total amount. It then executes a query to select distinct store codes from a fixed sales table.

```
CAST(quantity AS INTEGER) AS quantity,  
CAST(price AS DOUBLE) AS price,  
  
CAST(quantity AS INTEGER)  
* CAST(price AS DOUBLE)  
AS total_amount,  
  
CAST(sale_date AS DATE) AS sale_date  
  
FROM staging_sales  
===  
print("Transformation fixed successfully.")  
Transformation fixed successfully.  
  
[37]: display(  
    sql.execute("""  
        SELECT DISTINCT store_code  
        FROM est_sales_fixed  
        ORDER BY store_code  
        """).df()  
)
```

	store_code
0	ST001
1	ST002
2	ST003
3	ST004
4	ST005

Key Questions / Analysis / Interpretation

Que 1: How does the separation of compute and storage in modern cloud data lakes shift the architectural preference from classic ETL models toward ELT?

Modern cloud data platforms commonly separate data storage from compute resources. Raw data can therefore be stored centrally and processed using scalable compute resources when required.

In ETL, transformations are performed before loading the data into the target system. This requires application-side processing and may require movement of raw data to application compute resources.

In ELT, raw data is loaded first and transformations are performed inside the target database or warehouse. This allows the database engine to use its own query-processing and compute capabilities.

Therefore, separation of storage and compute can make ELT suitable for large-scale analytical workloads because raw data can be retained and transformation processing can be performed using scalable database compute.

Que 2: Compare the pipelines in terms of maintaining robust data lineage tracking and recovering from intermediate transformation bugs.

ETL performs transformations in application memory before loading the final result. If intermediate datasets are not explicitly saved, debugging a transformation problem may require the pipeline to be executed again.

ELT provides a persistent staging layer between raw and transformed data:

Raw Data



Staging



Transformation



Final Data

This makes it easier to inspect the original data and correct SQL transformation logic.

The experiment demonstrated this by intentionally introducing an incorrect store-code transformation and subsequently correcting it using the original staging_sales table.

Que 3: Evaluate how network resource costs shift when running transformation calculations inside application compute nodes versus inside database storage engines.

In an ETL architecture, raw data may need to move from storage to application compute nodes. After transformation, processed data may then be transferred to the target database.

In an ELT architecture, the raw data is first loaded into the database or warehouse and transformations are performed by the database engine.

Therefore, ELT can reduce unnecessary movement of large datasets between separate application and database environments when the database compute engine is located close to the stored data.

In this local experiment, network cost is minimal because all components operate on the same machine.

Final Analysis / Interpretation

The experiment successfully implemented and evaluated two data integration architectures: an ETL pipeline using Python and Pandas and an ELT pipeline using DuckDB SQL.

The ETL pipeline transformed the raw retail transaction data in application memory before loading the cleaned data into DuckDB. The ELT pipeline first loaded the raw records into a DuckDB staging table and then performed transformations using SQL.

Both pipelines processed the same dataset and produced logically equivalent cleaned results. Execution time was measured using the same local environment to provide a performance comparison.

The experiment also demonstrated differences in data lineage and transformation recovery. The ELT architecture retained the original data in the staging table, allowing transformation logic to be modified and executed again without recreating the raw input dataset.

The performance results obtained from the local benchmark should be interpreted according to the hardware, dataset size, implementation, and database configuration used during the experiment.

Final Conclusion

The practical successfully demonstrated the implementation and comparison of ETL and ELT data integration architectures using Python, Pandas, and DuckDB.

The ETL pipeline performed transformations in application memory before loading the processed data into DuckDB. The ELT pipeline loaded raw data into a staging table and performed transformations using the DuckDB analytical engine.

The experiment demonstrated that both architectures can produce equivalent cleaned datasets while differing in transformation location, data lineage, debugging, resource utilization, and maintenance requirements.

The ELT architecture provides a persistent staging layer that supports easier inspection and recovery from transformation errors. ETL provides a straightforward approach when transformation logic is naturally implemented in application code.

The appropriate architecture depends on factors such as dataset volume, infrastructure capabilities, compute location, network costs, transformation complexity, lineage requirements, and maintenance needs.